Rung 2 — Where the `x⁶` comes from (Taylor's theorem with a remainder)

Where you start: you've derived the `18/D` half of the certificate (Rung 1) and you can read a `∀` statement and follow a proof's moves (Rung 1.5). Now you close the loop: the *other* half of the bound, `x⁶`.

What you'll be able to do: explain why a piece of hardware that can only add and multiply can compute `sin` at all — and put a **proven** ceiling on how wrong that approximation is.

---

1. The bench

The certificate is `|sin_hw(x) − sin(x)| ≤ 18/D + x⁶`. Rung 1 gave you the `18/D` — the fixed-point truncation. But there's a second, deeper reason the hardware is wrong, and it's hiding in one line of `eml_sin.v`:

```
// sin(x) ~= x - x^3/6 + x^5/120
```

The hardware **does not compute `sin`.** No circuit can — `sin` isn't made of adds and multiplies. So `eml_sin` computes a *polynomial* that's close to `sin` near 0. The gap between that polynomial and the real `sin` is the `x⁶`. Our job: prove how big that gap can get.

2. The one new idea: a Taylor polynomial, and its leftover

Why a polynomial? Hardware can add and multiply. That's it. So the only functions it can build directly are polynomials. To compute anything else — `sin`, `exp`, `log` — you approximate it with a polynomial.

Which polynomial? The one that matches `sin` and its slopes at `x = 0`: the **Taylor polynomial** (at 0, also called Maclaurin). Match the value, the 1st derivative, the 2nd, ... For `sin` the first three nonzero terms are:

```
T(x) = x − x³/6 + x⁵/120
```

(Those `6` and `120` are `3!` and `5!` — the factorials Taylor's recipe puts there.) This is *exactly* the line in `eml_sin.v`.

The leftover is the remainder. Everything the polynomial misses is `R(x) = sin(x) − T(x)`. Rung 2's whole content is one theorem that bounds `R(x)`.

3. Do it by hand

Taylor's theorem (Lagrange form) says the remainder isn't a mystery — it looks just like "the next term," but with the derivative evaluated at some unknown point `ξ` (xi) between 0 and `x`:

```
R(x) = f⁽⁶⁾(ξ) / 6!   ·   x⁶        for some ξ between 0 and x
```

Here `f = sin`. Now bound it, step by step (this is a `∀`→`∃` argument, Rung 1.5 grammar):

1. The 6th derivative of `sin` is `-sin`. So `f⁽⁶⁾(ξ) = -sin(ξ)`.
2. `sin` of anything is at most 1 in size: `|sin(ξ)| ≤ 1`.
3. `6! = 720`. So:

```
|R(x)| = |sin(ξ)| / 720 · x⁶  ≤  x⁶ / 720  ≤  x⁶
```

That's it — that's the `x⁶`. The proof uses the clean, slightly loose ceiling `x⁶` (dropping the `/720`) because it's simpler to carry around and still true.

Sanity check with real numbers. At `x = 1`: the true miss is `sin(1) − T(1) = 0.841471 − 0.841667 ≈ −0.000196`. The proven ceiling `x⁶/720 = 1/720 ≈ 0.00139` holds (about 7× slack), and the certificate's even-looser `x⁶ = 1` holds trivially. At `x = 0.4` the ceiling is already `0.4⁶ ≈ 0.0041` — small. Near 0 it's microscopic.

4. Make it formal

Why is Taylor's theorem *true*? It's built from the **Mean Value Theorem (MVT)**: for a smooth function on an interval, there's some point where the *instantaneous* slope equals the *average* slope over the interval. (That "there's some point" is an ∃ — the one from Rung 1.5.) Apply the MVT once and you get the first-order remainder; apply it again to the remainder, and again, and after six passes you land on the `x⁶` form above. machlib builds exactly this — a *repeated-MVT chain* — from scratch:

```
MachLib.SinTaylorRemainder.R0_bound     -- |sin(x) - (x - x³/6 + x⁵/120)| ≤ x⁶  on [0,1]
```

No `sorry`, no borrowed axioms — the whole MVT chain is checked. This is the first place your Rung 1.5 practice pays off: you can now read that proof and see the `∀` / `∃` skeleton instead of a wall of symbols.

5. Watch it hold — and you've now derived the *whole* certificate

Put the two rungs together:

```
|sin_hw(x) − sin(x)|  ≤   18/D    +    x⁶
                          ^^^^^         ^^^
                     fixed-point     Taylor remainder
                      (Rung 1)         (Rung 2)
```

- Near `x = 0`, `x⁶` is negligible, so the flat `18/D ≈ 0.000275` dominates — which is why our capture's *tightest margin* showed up at small `x`.
- As `x → 1`, `x⁶` grows and takes over — which is why the *observed* hardware error crept up as `x` increased.

Both fingerprints are exactly what 1201 real samples showed. **You just re-derived, from scratch, the entire bound a research team proved in Lean** — and you understand why each piece is there.

The cliffhanger → Rung 3

One honest gap remains. You bounded the two error sources **separately**. But the real hardware does fixed-point arithmetic *and* the Taylor approximation *at the same time*, and the errors interact — an early rounding error gets fed through later multiplies, mixing with the approximation error. Bounding *that* — how errors **compose** through a computation — is the last piece, and it's the idea behind the composed `sin²+cos²` certificate you may have seen on the bench.

→ **Next:** Rung 3 — *Composing error bounds* — the triangle inequality and `qmul_err`, to turn your two separate ceilings into the real end-to-end proof. Then Rung 4: hand it to Lean.

Exercises & checkpoint

1. Compute the 3-term Taylor value `T(0.5) = 0.5 − 0.5³/6 + 0.5⁵/120` and compare it to `sin(0.5)`. Is the actual error below the ceiling `0.5⁶`? Below `0.5⁶/720`?
2. Explain why the certificate uses the ceiling `x⁶` rather than the tighter `x⁶/720`. What do you gain by being slightly loose?
3. Solve `x⁶ = 18/65536` for `x`. That's roughly where the Taylor error overtakes the fixed-point error — sanity-check it against where the observed error started climbing.

Checkpoint: you can explain why the hardware uses a polynomial, and where the `x⁶` comes from.